



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

پیش‌بینی پیوند^۱ در محیط خصمانه^۲!



سلام، به پروژه پایانی شبکه‌های پیچیده پویا خوش آمدید ❤️

در این پروژه، شما یک Pipeline کامل برای پیش‌بینی پیوند مقاوم در برابر حملات خصمانه طراحی خواهید کرد. هدف این نیست که فقط یک مدل آموزش^۳ دهید، بلکه هدف آن است که مانند یک پژوهشگر فکر کنید: **مدل بسازید، به آن حمله کنید و از آن دفاع کنید!**

سوال اصلی پروژه:

آیا می‌توان یک مدل پیش‌بینی پیوند ساخت که در برابر یک مهاجم^۴ با سرمایه محدود، پایدار بماند؟

🔥 پیش‌نیازها:

انتظار می‌رود در طول ترم و در کنار ارائه‌هایی که از سوی دانشجویان ارائه شده است، به صورت پایه، با مفاهیم زیر آشنا باشید:

- مفاهیم پایه گراف (که بارها در تمرین‌های طول ترم با آنها کار کرده‌اید)
- آشنایی با GNN و Message Passing
- آشنایی با PyTorch Geometric و PyTorch
- مفهوم پایه Link Prediction و معیارهای ارزیابی آن

در صورتی که با برخی از این مفاهیم آشنا نیستید، نگران نباشید! کافی است به نزدیکترین و در دسترس‌ترین سایت هوش مصنوعی مثل ChatGPT، Z.ai، Claude، DeepSeek، Manus و مراجعه کرده و از آن بخواهید تا تعاریف و مفاهیم اولیه هر کدام را برای شما توضیح دهد.

۱: استفاده از نمونه کدهای قرار داده شده در طول تمرین اختیاری است.

۲: کدها کامل نیستند و شما باید با ایده خودتان و مقالاتی که مطالعه خواهید کرد آن‌ها را پیاده‌سازی کنید.

۳: همچنین در بین کدها، گاهی سؤالاتی پرسیده شده است که نیاز است به آن‌ها پاسخ دهید

Train^۲
Attacker^۴

Link prediction^۱
Adversarial Environment^۲



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

📌 مقالات اجباری:

پیش از شروع پروژه، نیازمند آن هستید که به درک خوبی از مقالاتی که لیست شده برسید. سوالات پروژه به گونه ای طراحی شده‌اند که بدون درک این مقالات، قادر به پاسخگویی نخواهید بود.

همانطور که اکثر شما دانشجویان، در این ترم یا در ترم های گذشته خود، درس سمینار را داشته‌اید، پیشنهاد می‌کنم از تکنیک‌هایی که برای خوانش مقالات به شما آموخته شده است استفاده کنید تا نیاز نباشد خط به خط مقالات پیشنهادی را مطالعه کنید.

اولویت	مقاله	چرا باید بخوانید؟
اجباری	لینک	مکانیزم اصلی حمله در این مقاله توضیح داده شده است.
اجباری	لینک	از اصلی ترین روش های دفاعی را بیان می‌کند.
اجباری	لینک	مدل Baseline شماست ! باید بدانید چرا SEAL برای Link Prediction انتخاب می‌شود نه GCN ساده؟
مهم	لینک	رویکرد جایگزین برای Defense
مهم	لینک	یک Library که بسیاری از حملات و دفاع ها را پیاده سازی کرده است

حتما باید مقالات "اجباری" را مطالعه نمایید. همچنین مقالاتی که با عنوان "مهم" نام برده شده است، به شما کمک خواهند کرد تا با دیدگاه بهتری به استقبال از سوالات بروید.

📌 دیتاست:

👉 دیتاست اصلی: Cora

Cora یک شبکه استناد علمی یا همان Citation Network است که در تحقیقات گراف بسیار مورد استفاده قرار می‌گیرد.

این دیتاست از طریق PyTorch Geometric به صورت خودکار دانلود می‌شود و نیازی به دانلود دستی نیست.

```
from torch_geometric.datasets import Planetoid
dataset = Planetoid(root='./data', name='Cora')
```



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

📁 دیتاست دوم: Citeseer

این دیتاست هم از طریق PyTorch Geometric به صورت خودکار دانلود می‌شود و نیازی به دانلود دستی نیست.

از این دیتاست برای تست تعمیم پذیری استفاده خواهید کرد.

```
from torch_geometric.datasets import Planetoid
dataset = Planetoid(root='./data', name='CiteSeer')
```

شما باید نتایج را روی هر دو دیتاست گزارش دهید و تفاوت رفتار مدل را توضیح دهید.

پیشنهاد می‌کنم به [لینک](#) مراجعه کنید و در ارتباط با این کتابخانه و دیتاست ها مطالعه کنید.

🧨 ساختار پیشنهادی پروژه:

```
project/
├── data/                                # Your Dataset Must be here
├── phase1_baseline/
│   ├── model.py                        # GNN for LinkPrediction
│   ├── train.py
│   └── evaluate.py
├── phase2_attack/
│   ├── attacker.py                    # Attack Implementation
│   └── evaluate_attack.py
├── phase3_defense/
│   ├── robust_model.py                # Robust Model
│   └── evaluate_defense.py
├── phase4_analysis/
│   └── analysis.ipynb
└── report.pdf
```

اجباری برای استفاده از ساختار پیشنهادی نیست. شما آزادید از هر ساختاری که مناسب می‌دانید استفاده کنید. تنها شرط الزامی آن است که خروجی نهایی بسیار تمیز و مرتب باشد و عملکرد هر پوشه به روشنی قابل تشخیص باشد.



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

مرحله اول: ساخت Baseline 🧠

قبل از اینکه بتوانید از چیزی دفاع کنید، باید بدانید از چه چیزی دارید دفاع میکنید!

در این مرحله یک مدل پیش‌بینی پیوند خواهید ساخت.

قبل از پیاده سازی، با ارجاع به مقالاتی که در قسمت ابتدایی خوانده‌اید و با رفرنس دادن بسیار دقیق به آن، بیان کنید که چرا برای Link Prediction از GCN ساده استفاده نمی‌کنیم؟ تفاوت اساسی مقاله ای که از SEAL استفاده کرده ای با رویکرد encode-then-decode چیست؟

👉 آماده سازی داده برای Link Prediction

برخلاف دسته‌بندی نود^۵، در پیش‌بینی پیوند باید یال‌ها را به سه قسمت تقسیم کنید:

- Train
- Val
- Test

```
from torch_geometric.transforms import RandomLinkSplit

transform = RandomLinkSplit(
    num_val=0.05,
    num_test=0.1,
    is_undirected=True,
    add_negative_train_samples=False, # Why FALSE? Explain please
    neg_sampling_ratio=1.0
)

train_data, val_data, test_data = transform(data)
```

چرا در تقسیم بندی داده ها برای پیش‌بینی پیوند، باید مراقب نشت داده^۶ باشیم؟ اگر یال‌های آزمون^۷، هنگام آموزش^۸ در ماتریس مجاورت^۹ باشند، چه اتفاقی خواهد افتاد؟ کامل توضیح دهید.

Train^۸
Adjacency Matrix^۹

Node Classification^۵
Data Leakage^۶
Test^۷



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

📁 پیاده سازی مدل

در این مرحله باید یک GNN دو لایه برای تولید Node Embedding بسازید. سپس، برای هر جفت نود، احتمال وجود یال را پیشبینی کنید.

```
import torch
import torch.nn.functional as F
from torch_geometric.nn import GCNConv

class LinkPredGNN(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def encode(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        x = F.dropout(x, p=0.5, training=self.training)
        return self.conv2(x, edge_index)

    def decode(self, z, edge_label_index):
        # What is happening in this function? EXPLAIN
        src = z[edge_label_index[0]]
        dst = z[edge_label_index[1]]
        return (src * dst).sum(dim=-1)

    def forward(self, x, edge_index, edge_label_index):
        z = self.encode(x, edge_index)
        return self.decode(z, edge_label_index)
```



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

Training Loop 📁

```
from sklearn.metrics import roc_auc_score, average_precision_score

def train(model, optimizer, train_data):
    model.train()
    optimizer.zero_grad()

    out = model(train_data.x,
                  train_data.edge_index,
                  train_data.edge_label_index)

    loss = F.binary_cross_entropy_with_logits(
        out, train_data.edge_label.float()
    )
    loss.backward()
    optimizer.step()
    return loss.item()

@torch.no_grad()
def evaluate(model, data):
    model.eval()
    out = model(data.x, data.edge_index, data.edge_label_index)
    pred = out.sigmoid().cpu()
    target = data.edge_label.cpu()

    auc = roc_auc_score(target, pred)
    ap = average_precision_score(target, pred)
    return auc, ap
```

توضیح دهید چرا از AUC-ROC و Average Precision به عنوان معیار استفاده می‌کنیم و نه Accuracy؟

مدل شما باید روی دیتاست Cora به این نتایج، در بدترین شرایط برسد:

```
AUC ≥ 0.88
AP ≥ 0.88
```

اگر به این اعداد نرسیدید، قبل از رفتن به قسمت بعدی، مشکل را رفع کنید.

یک Baseline ضعیف، حمله و دفاع را بی معنی خواهد کرد!



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

مرحله دوم: طراحی و پیاده‌سازی مهاجم^{۱۰}

برای این مرحله شما باید مقاله مربوط به آن را (Nettack) خوانده باشید.

حال نوبت آن است که از موضوع یک مهاجم فکر کنید. شما به مدل آموزش داده شده دسترسی دارید و می‌خواهید با کمترین تغییر ممکن در گراف، بیشترین آسیب را وارد کنید.

هدف مهاجم:

- یال‌هایی که باید پیش‌بینی شوند را معین کنید.
- با اضافه یا حذف کردن حداکثر k یال از گراف ($\text{Budget} = k$)، مدل را مجبور کنید این یال‌ها را اشتباه پیش‌بینی کند.

محدودیت‌های مهاجم:

- بودجه: $k=5\%$ تعداد کل یال‌ها
- تغییرات نباید خیلی آشکار باشند (unnoticeability)
- فقط می‌توان یال اضافه یا حذف کرد (نه نود)

سوالاتی که به خوبی و با تسلط بر مقاله Nettack باید به آن پاسخ دهید این است که:

در مقاله نام برده، حمله برای Node Classification طراحی شده است. وقتی می‌خواهیم آن را برای پیش‌بینی پیوند استفاده کنیم، هدف حمله و پیاده‌سازی آن، چطور تغییر خواهد کرد؟



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

شما باید دو نوع حمله را پیاده سازی کنید:

(Baseline) Random Attack 📁

```
import random

def random_attack(edge_index, num_nodes, budget, seed=42):

    random.seed(seed)
    existing_edges = set(map(tuple, edge_index.t().tolist()))

    new_edges = []
    attempts = 0
    while len(new_edges) < budget and attempts < budget * 100:
        u = random.randint(0, num_nodes - 1)
        v = random.randint(0, num_nodes - 1)
        if u != v and (u, v) not in existing_edges:
            new_edges.append([u, v])
            existing_edges.add((u, v))
            attempts += 1

    return torch.tensor(new_edges).t()
```




دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

Gradient-based Attack 📁

```
def compute_attack_score(model, data, candidate_edge, target_edges):
    """
    For each candidate edge, compute how much adding it
    would damage the model's prediction on target edges.

    This function must be completed by the student.
    Hint: use gradient information or a surrogate objective.
    """
    # TODO: implement this function
    # 1. Add the candidate edge to edge_index
    # 2. Run a forward pass through the model
    # 3. Measure how much the model's confidence on target edges drops
    # 4. Return that score
    pass

def greedy_attack(model, data, target_edges, budget):
    """
    At each step, adds the edge that causes the most damage.

    This is greedy – why is it not optimal?
    """
    perturbed_edge_index = data.edge_index.clone()

    for step in range(budget):
        best_edge = None
        best_score = -float('inf')

        # TODO: define the search space
        # Note: exhaustive search over all possible edges
        # is not feasible for large graphs – why?
        # What do you do instead?

        candidates = sample_candidates(data, n=500)

        for candidate in candidates:
            score = compute_attack_score(
                model, data, candidate, target_edges
            )
            if score > best_score:
                best_score = score
                best_edge = candidate

        # Add the best edge found in this step
        perturbed_edge_index = add_edge(perturbed_edge_index, best_edge)

    return perturbed_edge_index
```

الگوریتم Greedy Attack که پیاده سازی کردید، در هر مرحله بهترین یال را اضافه میکند. این رویکرد چه مشکل اساسی دارد؟ چرا بهینه نیست؟



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

در تعریف مسئله گفتیم که حمله باید unnoticeable باشد. یک معیار کمی برای سنجش این موضوع تعریف کنید و نشان دهید که حمله شفاف این معیار را نقص نمی‌کند.

👉 ارزیابی حمله

```
def evaluate_attack(model, clean_data, perturbed_edge_index, target_edges):
    auc_clean, ap_clean = evaluate_on_targets(
        model, clean_data, target_edges
    )

    perturbed_data = clean_data.clone()
    perturbed_data.edge_index = perturbed_edge_index
    auc_attacked, ap_attacked = evaluate_on_targets(
        model, perturbed_data, target_edges
    )

    attack_success_rate = (auc_clean - auc_attacked) / auc_clean

    return {
        'auc_clean': auc_clean,
        'auc_attacked': auc_attacked,
        'attack_success_rate': attack_success_rate
    }
```

نتایج حمله را روی نودهایی با درجه بالا در مقابل نودهایی با درجه پایین مقایسه کنید. آیا نودهایی با درجه کم، آسیب پذیر تر هستند؟ آیا این انتظار منطقی است؟ نتیجه تجربی شما این فرضیه را تایید می‌کند؟

🔴 مرحله سوم: ساخت مدل Robust (Defense)

حال که می‌دانید مدل چطور آسیب می‌بیند، باید آن را مقاوم کنید. در این مرحله دو روش دفاعی را پیاده‌سازی خواهید کرد.

قبل از این مرحله باید مقاله GNN-Guard را خوانده باشید!

👉 روش اول: GNN-Guard

ایده اصلی بیان می‌کند یال‌هایی که توسط مهاجم اضافه شده‌اند، معمولاً بین گره‌هایی هستند که ویژگی مشابهی ندارند. GNN-Guard به این یال‌ها در مرحله Aggregation، وزن کمتری اختصاص می‌دهد.

مقاله GNN-Guard در بستر Node Classification ارائه شده است، اما مکانیزم دفاعی آن، یعنی وزن‌دهی به یال‌ها بر اساس شباهت ویژگی گره‌ها، در سطح Aggregation گراف عمل می‌کند. بخشی از ارزیابی شما به این است که بتوانید این انتقال از Node Classification به Link Prediction را به صورت مستدل توضیح دهید.



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

```
import torch
import torch.nn.functional as F
from torch_geometric.nn import MessagePassing

class GNNGuardConv(MessagePassing):
    def __init__(self, in_channels, out_channels, threshold=0.1):
        super().__init__(aggr='add')
        self.linear = torch.nn.Linear(in_channels, out_channels)
        self.threshold = threshold

    def forward(self, x, edge_index):
        # Compute feature similarity for each edge
        row, col = edge_index

        # TODO: compute cosine similarity between features of the two endpoint nodes of each edge
        similarity = self.compute_similarity(x[row], x[col])

        # TODO: assign low or zero weight to edges whose similarity is below the threshold
        edge_weight = self.compute_edge_weight(similarity)

        return self.propagate(edge_index, x=x, edge_weight=edge_weight)

    def compute_similarity(self, x_src, x_dst):
        # TODO: implement this method
        # Hint: use cosine similarity or normalized dot product
        pass

    def compute_edge_weight(self, similarity):
        # TODO: implement this method
        # Hint: edges with low similarity should receive low weight
        pass

    def message(self, x_j, edge_weight):
        return edge_weight.view(-1, 1) * x_j

    def update(self, aggr_out):
        return self.linear(aggr_out)
```

روش دوم: Adversarial Training 📁

ایده اصلی بیان می‌کند که در حین آموزش، مدل را روی نسخه‌های مسموم شده گراف هم باید آموزش داد تا یاد بگیرد در برابر آشفتگی^{۱۱} مقاوم باشد.

^{۱۱} Perturbation



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

```
def adversarial_training(model, optimizer, train_data, attack_fn, adv_budget=10, adv_weight=0.5):  
    """  
    Each epoch trains on both the clean data and the poisoned data.  
    """  
    model.train()  
    optimizer.zero_grad()  
  
    # Loss on clean data  
    out_clean = model(train_data.x,  
                       train_data.edge_index,  
                       train_data.edge_label_index)  
    loss_clean = F.binary_cross_entropy_with_logits(  
        out_clean, train_data.edge_label.float()  
    )  
  
    # Generate the poisoned version of the graph  
    with torch.no_grad():  
        perturbed_edge_index = attack_fn(  
            model, train_data, budget=adv_budget  
        )  
  
    # Loss on poisoned data  
    out_adv = model(train_data.x,  
                    perturbed_edge_index, # poisoned graph  
                    train_data.edge_label_index)  
    loss_adv = F.binary_cross_entropy_with_logits(  
        out_adv, train_data.edge_label.float()  
    )  
  
    # Combined loss  
    loss = (1 - adv_weight) * loss_clean + adv_weight * loss_adv  
    loss.backward()  
    optimizer.step()  
  
    return loss.item()
```

بعد از اعمال دفاع ^{۱۲}، مدل شما باید دو شرط را همزمان برآورده کند. اول اینکه روی داده تمیز ^{۱۳} نسبت به baseline بیشتر از ۳٪ افت AUC نداشته باشد، و دوم اینکه بعد از حمله (از مرحله ۲)، افت AUC از ۱۰٪ تجاوز نکند.

اگر مدل شما روی داده تمیز عملکرد خیلی ضعیف‌تری نسبت به baseline داشت، نشانه آن است که دفاع به جای یادگیری مقاومت کلی، صرفاً روی همان حمله خاص بیش‌برازش ^{۱۴} شده است.

^{۱۴} Overfeat

^{۱۲} Defense

^{۱۳} Clean



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

مرحله چهارم: تحلیل نهایی

در گزارش نهایی خود باید این جدول را کامل کنید:

دیتاست Cora				
Attack Success Rate	AP	AUC	داده	مدل
×			Clean	Baseline GNN
			After Random Attack	Baseline GNN
×			Clean	GNN-Guard
			After Gradient Attack	GNN-Guard
×			Clean	Adversarial Training
			After Gradient Attack	Adversarial Training

دیتاست Citeseer				
Attack Success Rate	AP	AUC	داده	مدل
×			Clean	Baseline GNN
			After Random Attack	Baseline GNN
×			Clean	GNN-Guard
			After Gradient Attack	GNN-Guard
×			Clean	Adversarial Training
			After Gradient Attack	Adversarial Training

رفتار مدل و مهاجم را روی هر دو دیتاست اجرا شده، تحلیل کنید.

یال‌هایی که مهاجم در مرحله ۲ اضافه کرده بود را بررسی کنید. آیا الگویی دارند؟

- بیشتر بین نودهایی با درجه بالا است یا پایین؟
- آیا معمولا بین نودهایی از کلاس‌های مختلف است؟



دانشگاه اصفهان

درس شبکه‌های پیچیده پویا

پروژه پایانی

استاد: دکتر افسانه فاطمی

📌 تذکرات بسیار بسیار مهم:

۱: همانند تمرین‌های قبلی، نمره کامل به کسانی اختصاص خواهد یافت که این تمرین و نتایج آن را از ابعاد مختلف مورد بررسی قرار دهند و تحلیل‌های کامل و با جزئیات ارائه دهند. نمایش دادن چند عدد در خروجی و پر کردن جدول‌های داده شده نمره خاصی نخواهد داشت.

۲: پروژه شما ارائه آنلاین/حضوری خواهد داشت و در آن شما باید به صورت کامل تمام Pipeline اجرا شده خود را توضیح دهید و در خصوص کدهای زده شده توضیحات کامل ارائه دهید.

۳: استفاده از کتابخانه DeepRobust آزاد است اما باید شناخت کافی نسبت به آن داشته باشید.

۴: پیشنهاد می‌کنم از Google Colab و GPU استفاده کنید.

۵: همه آزمایش‌ها باید با `seed=42` اجرا شوند.

در صورت هرگونه سوال، ابهام، ایراد، راهنمایی و در تلگرام (@alimoeinian) و در بله (@ali_hates_bale) با من در ارتباط باشید.